

A Representative LaTeX Benchmark Document

Scribe Benchmark Suite

May 23, 2026

Abstract

This document exercises a realistic subset of LaTeX features used in academic papers: sectioning, cross-references, tables, lists, theorems, math display, code listings, citations, and tables of contents. The goal is to measure compile time on a workload representative of what Scribe users actually run.

Contents

1	Introduction	1
1.1	Motivation	1
1.2	Contributions	2
2	Background	2
3	Methods	2
3.1	Setup	2
3.2	Hardware	2
3.3	Code Listing	2
4	Results	3
5	Discussion	3
6	Conclusion	3

1 Introduction

Modern academic writing relies on $\text{\LaTeX}$ for typesetting equations and managing references. This benchmark measures wall-clock time across two pipelines: Tectonic (standalone) and pandoc-via-pdflatex (MiKTeX). See Section 4 for a discussion of the trade-offs.

1.1 Motivation

The compile path is the slowest user-facing operation in Scribe. Reducing it has a direct effect on perceived responsiveness.

1.2 Contributions

- A reproducible benchmark workload.
- Cold-cache and warm-cache timings for both engines.
- Recommendations for the default Scribe compile pipeline.

2 Background

Theorem 1 (Pythagoras). *For any right triangle with legs a and b and hypotenuse c ,*

$$a^2 + b^2 = c^2.$$

Lemma 2. *If $x = y$ then $y = x$.*

A more elaborate display:

$$\mathcal{L}(\theta) = -\frac{1}{N} \sum_{i=1}^N \log p(y_i \mid x_i; \theta) + \lambda \|\theta\|_2^2 \quad (1)$$

$$\nabla_{\theta} \mathcal{L} = -\frac{1}{N} \sum_{i=1}^N \nabla_{\theta} \log p(y_i \mid x_i; \theta) + 2\lambda\theta. \quad (2)$$

3 Methods

3.1 Setup

We measure five consecutive compiles of an identical source. The first compile of each run pays the cold cache cost (package downloads for Tectonic, font cache initialisation for MiKTeX). Subsequent compiles exercise the warm path. We report median wall time across runs two through five.

3.2 Hardware

Component		Value
CPU	x86_64	(Windows 11)
RAM		As-detected
Storage		NVMe SSD

Table 1: Test host configuration.

3.3 Code Listing

```
def loss(params, batch):
    logits = forward(params, batch.x)
    return cross_entropy(logits, batch.y)
```

Engine	Cold (s)	Warm (s)
Tectonic	TBD	TBD
pandoc + pdflatex	TBD	TBD

Table 2: Compile-time measurements (lower is better).

4 Results

Table 2 summarises the timings, populated after the benchmark completes.

5 Discussion

Theorem 1 reminds us that geometry is a thing. Lemma 2 is also true.

6 Conclusion

See Section 4.